



طراحی سامانه‌های رقمی

نیم‌سال دوم ۰۴-۰۵

استاد: جناب آقای دکتر اجلالی

مهلت پاسخ: ۵ مرداد ماه

Verilog Programming Language

تمرین پنجم

- مهلت ارسال پاسخ تمرین تا ساعت ۲۳:۵۹ روز تعیین شده در سامانه آموزشی درس است.
- هم‌فکری در انجام تمرین مجاز است؛ اما پاسخ نهایی هر دانشجو باید توسط خود او نوشته شده باشد. در صورت استفاده از هم‌فکری یا منابع خارج از درس، نام افراد و منابع ذکر شود.
- تمام کدهای تمرین باید با زبان برنامه‌نویسی Verilog نوشته شوند و خوانا، پیمانه‌ای^۱ و دارای کامنت انگلیسی باشند.
- برای هر سؤال، یک میز آزمون^۲ مستقل نوشته شود که حالت‌های عادی، مرزی و خطاهای احتمالی را پوشش دهد.
- برای هر سؤال، بخشی جداگانه در گزارش ارائه شود که شامل توضیح معماری، توضیح کد، تحلیل رفتار مدار، خروجی شبیه‌سازی و موج‌نگار^۳ باشد.
- پاسخ نهایی باید شامل کدهای منبع^۴، میزهای آزمون، خروجی‌های شبیه‌سازی، تصاویر موج‌نگار و گزارش کامل باشد.
- فایل نهایی را به صورت فشرده zip با قالب DSD_HW5_[firstName]_[lastName]_[StudentId] در سامانه آموزشی درس (CW) بارگذاری کنید.

^۱Modular

^۲Testbench

^۳Waveform

^۴Source Code

سؤال اول: طراحی ضرب کننده اعداد مختلط (۳۰ نمره)

پیمانه‌ای طراحی کنید که در هر سیکل ساعت، دو عدد مختلط را به صورت هم‌زمان دریافت کند. عدد مختلط اول با نام A شامل بخش حقیقی A_r و بخش موهومی A_i است. عدد مختلط دوم با نام B شامل بخش حقیقی B_r و بخش موهومی B_i است. هر یک از این چهار بخش از طریق یک گذرگاه ورودی ۱۵ بیتی به مدار داده می‌شود.

سیگنال کنترلی $conj$ تعیین می‌کند که عملیات ضرب به چه شکلی انجام شود. اگر $conj = 0$ باشد، مدار باید حاصل ضرب معمولی دو عدد مختلط را محاسبه کند:

$$A \times B$$

اگر $conj = 1$ باشد، مدار باید عدد A را در مزدوج مختلط^۵ عدد B ضرب کند:

$$A \times B^*$$

خروجی مدار شامل دو گذرگاه ۳۱ بیتی است که به ترتیب بخش حقیقی و بخش موهومی حاصل ضرب را نشان می‌دهند. تمام ورودی‌ها و خروجی‌ها باید به صورت علامت‌دار و در نمایش مکمل دو^۶ تفسیر شوند.

مشخصه‌ها و رابط‌های ورودی و خروجی

نام مشخصه	مقدار پیشنهادی	نوع	توضیح
IN_WIDTH	۱۵	عدد صحیح	پهنای هر بخش حقیقی یا موهومی ورودی
OUT_WIDTH	۳۱	عدد صحیح	پهنای هر بخش حقیقی یا موهومی خروجی

پیمانه سطح بالا باید دارای رابط زیر باشد:

نام پورت	جهت	پهنای	توضیح
clk	ورودی	۱ بیت	سیگنال ساعت مدار
rst	ورودی	۱ بیت	بازنشانی هم‌گاه و فعال-بالا
A_r	ورودی	۱۵ بیت	بخش حقیقی عدد A ، به صورت علامت‌دار
A_i	ورودی	۱۵ بیت	بخش موهومی عدد A ، به صورت علامت‌دار
B_r	ورودی	۱۵ بیت	بخش حقیقی عدد B ، به صورت علامت‌دار
B_i	ورودی	۱۵ بیت	بخش موهومی عدد B ، به صورت علامت‌دار
conj	ورودی	۱ بیت	انتخاب نوع ضرب؛ صفر برای $A \times B$ و یک برای $A \times B^*$
Y_r	خروجی	۳۱ بیت	بخش حقیقی حاصل ضرب، به صورت علامت‌دار
Y_i	خروجی	۳۱ بیت	بخش موهومی حاصل ضرب، به صورت علامت‌دار

رفتار مورد انتظار برنامه

^۵Complex Conjugate

^۶Two's Complement

۱. در هر لبه بالارونده ساعت، ورودی‌های A_i, B_r, B_i و $conj$ ابتدا در ثبات‌های^۷ داخلی ذخیره شوند.
۲. خروجی‌ها باید بر اساس مقادیر ذخیره‌شده در ثبات‌ها محاسبه شوند، نه مستقیماً بر اساس ورودی‌های لحظه‌ای.
۳. در حالت $conj = 0$ ، روابط زیر باید پیاده‌سازی شوند:

$$Y_r = A_r B_r - A_i B_i$$

$$Y_i = A_r B_i + A_i B_r$$

۴. در حالت $conj = 1$ ، روابط زیر باید پیاده‌سازی شوند:

$$Y_r = A_r B_r + A_i B_i$$

$$Y_i = A_i B_r - A_r B_i$$

۵. حاصل ضرب دو عدد ۱۵ بیتی علامت‌دار، ۳۰ بیت نیاز دارد. جمع یا تفریق دو حاصل ضرب ۳۰ بیتی می‌تواند به ۳۱ بیت نیاز داشته باشد؛ بنابراین خروجی ۳۱ بیتی برای این طراحی کافی و مناسب است.
۶. با فعال شدن rst در لبه بالارونده ساعت، تمام ثبات‌های ورودی و خروجی باید صفر شوند.

مطلوب است

۱. پیمان‌ه ضرب‌کننده مختلط را با زبان برنامه‌نویسی Verilog طراحی کنید.
۲. یک میز آزمون کامل بنویسید که دست‌کم حالت‌های زیر را بررسی کند.
 - (آ) ضرب دو عدد مختلط با مؤلفه‌های مثبت؛
 - (ب) ضرب دو عدد مختلط با مؤلفه‌های منفی؛
 - (ج) حالت $conj = 0$ ؛
 - (د) حالت $conj = 1$ ؛
 - (ه) ورودی صفر؛
 - (و) مقادیر مرزی مثبت و منفی ۱۵ بیتی؛
۳. خروجی متنی شبیه‌سازی، تصویر موج‌نگار و گزارش توضیح کد و نتایج را ارائه کنید.

⁷Registers

سؤال دوم: طراحی کنترل کننده چراغ راهنمایی چهارراه (۳۰ نمره)

یک چهارراه شامل دو مسیر عبور شمال-جنوب و شرق-غرب را در نظر بگیرید. هدف این سؤال، طراحی یک سامانه رقمی^۸ هم‌گاه^۹ برای کنترل چراغ‌های راهنمایی این چهارراه با استفاده از زبان Verilog است. طراحی باید به صورت پیمانه‌ای انجام شود و در آن از توصیف ساختاری^{۱۰} و نمونه‌سازی پیمانه‌ها^{۱۱} استفاده گردد.

برای هر مسیر عبور، یک پیمانه مستقل چراغ راهنمایی در نظر گرفته می‌شود. این پیمانه با دریافت فرمان کنترلی از پیمانه اصلی، دقیقاً یکی از سه خروجی زیر را فعال می‌کند:

green, yellow, red

در این سؤال، مستقل بودن پیمانه چراغ به این معنا است که برای هر مسیر عبور، یک نمونه جداگانه از پیمانه چراغ ساخته می‌شود؛ اما تصمیم‌گیری نهایی درباره وضعیت هر مسیر باید توسط پیمانه اصلی کنترل چهارراه انجام شود. بنابراین دو پیمانه چراغ نباید خودمختار و بدون هماهنگی با یکدیگر تغییر وضعیت دهند.

مشخصه‌ها و رابط‌های ورودی و خروجی

نام مشخصه	مقدار پیشنهادی	نوع	توضیح
GREEN_TIME	قابل تنظیم	عدد صحیح	تعداد سیکل‌های ماندن هر مسیر در وضعیت سبز
YELLOW_TIME	قابل تنظیم	عدد صحیح	تعداد سیکل‌های ماندن هر مسیر در وضعیت زرد
ALL_RED_TIME	قابل تنظیم	عدد صحیح	تعداد سیکل‌های وضعیت ایمن که در آن هر دو مسیر قرمز هستند

پیمانه سطح بالا باید دارای رابط زیر باشد:

نام پورت	جهت	پهنای	توضیح
clk	ورودی	۱ بیت	سیگنال ساعت مدار
rst	ورودی	۱ بیت	بازنشانی هم‌گاه و فعال-بالا
start	ورودی	۱ بیت	سیگنال شروع کنترل چرخه‌ای چراغ‌ها
ns_green	خروجی	۱ بیت	چراغ سبز مسیر شمال-جنوب
ns_yellow	خروجی	۱ بیت	چراغ زرد مسیر شمال-جنوب
ns_red	خروجی	۱ بیت	چراغ قرمز مسیر شمال-جنوب
ew_green	خروجی	۱ بیت	چراغ سبز مسیر شرق-غرب
ew_yellow	خروجی	۱ بیت	چراغ زرد مسیر شرق-غرب
ew_red	خروجی	۱ بیت	چراغ قرمز مسیر شرق-غرب

پیمانه داخلی چراغ راهنمایی نیز می‌تواند دارای رابط زیر باشد:

^۸Digital

^۹Synchronous Digital System

^{۱۰}Structural Description

^{۱۱}Module Instantiation

نام پورت	جهت	پهنا	توضیح
light_state	ورودی	۲ بیت	فرمان وضعیت چراغ؛ قرمز، زرد یا سبز
green	خروجی	۱ بیت	خروجی چراغ سبز
yellow	خروجی	۱ بیت	خروجی چراغ زرد
red	خروجی	۱ بیت	خروجی چراغ قرمز

رفتار مورد انتظار

۱. پس از فعال شدن rst، هر دو مسیر باید در وضعیت ایمن قرار گیرند؛ یعنی هر دو چراغ قرمز باشند.
۲. اگر $start = 0$ باشد، سامانه باید در وضعیت ایمن باقی بماند.
۳. پس از فعال شدن start، کنترل چراغ‌ها باید به صورت چرخه‌ای انجام شود.
۴. هرگز نباید دو مسیر به طور هم‌زمان چراغ سبز داشته باشند.
۵. هنگامی که مسیر شمال-جنوب سبز است، مسیر شرق-غرب باید قرمز باشد.
۶. هنگامی که مسیر شرق-غرب سبز است، مسیر شمال-جنوب باید قرمز باشد.
۷. پیش از تغییر مسیر عبور، مسیر فعلی باید به مدت YELLOW_TIME سیکل در وضعیت زرد قرار گیرد.
۸. در زمان زرد بودن یک مسیر، مسیر مقابل باید همچنان قرمز باشد.
۹. بین پایان وضعیت زرد یک مسیر و شروع وضعیت سبز مسیر مقابل، می‌توان از وضعیت ALL_RED استفاده کرد تا گذار ایمن‌تر باشد.

مطلوب است

۱. پیمانۀ چراغ راهنمایی هر مسیر را طراحی کنید؛ به گونه‌ای که دقیقاً یکی از خروجی‌های green، yellow یا red در هر لحظه فعال باشد.
۲. پیمانۀ اصلی کنترل‌کننده چهارراه را با ماشین حالت متناهی^{۱۲}، شمارنده زمان و منطق تولید فرمان طراحی کنید.
۳. در پیمانۀ اصلی، دو نمونه جداگانه از پیمانۀ چراغ راهنمایی برای مسیرهای شمال-جنوب و شرق-غرب نمونه‌سازی کنید.
۴. مشخصه‌های GREEN_TIME، YELLOW_TIME و ALL_RED_TIME را به گونه‌ای تعریف کنید که برای شبیه‌سازی بتوان مقادیرهای کوچک و برای پیاده‌سازی مقادیرهای واقعی‌تر انتخاب کرد.
۵. یک میز آزمون کامل بنویسید که دست‌کم حالت‌های زیر را بررسی کند.
 - (آ) بازنشانی مدار؛
 - (ب) باقی ماندن در وضعیت ایمن هنگام $start = 0$ ؛
 - (ج) شروع چرخه پس از $start = 1$ ؛

¹²Finite State Machine, FSM

(د) گذار از سبز به زرد و سپس به قرمز؛

(ه) گذار از مسیر شمال-جنوب به شرق-غرب؛

(و) گذار از مسیر شرق-غرب به شمال-جنوب؛

(ز) بررسی این که دو خروجی سبز هرگز همزمان ۱ نمی شوند؛

(ح) بررسی این که در هر مسیر، بیش از یک چراغ همزمان روشن نمی شود؛

(ط) بررسی مقدارهای مرزی مشخصه‌ها در شبیه‌سازی، مانند مقدار ۱ برای زمان زرد یا سبز.

۶. خروجی متنی شبیه‌سازی، تصویر موج‌نگار، جدول حالت کنترل‌کننده و گزارش توضیح کد و نتایج را ارائه و تحلیل کنید.

سؤال سوم: طراحی صف هم‌گاه FIFO (۴۵ نمره)

در این سؤال باید یک صف با سیاست اولین ورودی-اولین خروجی^{۱۳} طراحی کنید. این مدار یک میانگیر داده^{۱۴} است که داده‌ها را به همان ترتیبی که وارد شده‌اند، از خروجی خارج می‌کند. صف دارای دو رابط اصلی است: رابط نوشتن^{۱۵} و رابط خواندن^{۱۶}. در این سؤال، هر دو عملیات خواندن و نوشتن با یک سیگنال ساعت مشترک انجام می‌شوند؛ بنابراین مدار موردنظر یک صف هم‌گاه است.

صف باید با استفاده از یک میانگیر چرخشی^{۱۷} و دو اشاره‌گر^{۱۸} پیاده‌سازی شود. این میانگیر می‌تواند به‌صورت یک آرایه ثباتی دوبعدی در Verilog تعریف شود. اشاره‌گر خواندن و اشاره‌گر نوشتن خانه‌های داخلی میانگیر را آدرس‌دهی می‌کنند و به‌ترتیب محل خواندن و نوشتن بعدی را مشخص می‌نمایند.

برای تشخیص وضعیت پر یا خالی بودن صف، تنها مقایسه ساده دو اشاره‌گر کافی نیست؛ زیرا برابر بودن اشاره‌گر خواندن و نوشتن می‌تواند هم نشان‌دهنده خالی بودن صف و هم، در برخی پیاده‌سازی‌ها، نشان‌دهنده پر بودن آن باشد. بنابراین باید از یک روش دیگری همانند شمارنده تعداد خانه‌های پر شده یا اشاره‌گرهای توسعه‌یافته با یک بیت اضافه استفاده شود.

مشخصه‌ها و رابط‌های ورودی و خروجی

نام مشخصه	مقدار پیش‌فرض	نوع	توضیح
DATA_WIDTH	۸	عدد صحیح	تعداد بیت‌های هر داده ورودی و خروجی
FIFO_DEPTH	۱۶	عدد صحیح	تعداد خانه‌های داخلی صف
ADDR_WIDTH	۴	عدد صحیح	تعداد بیت‌های لازم برای آدرس‌دهی خانه‌های صف

پیمانه سطح بالا باید دارای رابط زیر باشد:

نام پورت	جهت	پهنای	توضیح
clk	ورودی	۱ بیت	سیگنال ساعت مشترک برای خواندن و نوشتن
rst	ورودی	۱ بیت	بازنشانی هم‌گاه و فعال-بالا
din	ورودی	DATA_WIDTH بیت	داده ورودی برای نوشتن در صف
wr_en	ورودی	۱ بیت	فرمان نوشتن؛ فعال-بالا
rd_en	ورودی	۱ بیت	فرمان خواندن؛ فعال-بالا
dout	خروجی	DATA_WIDTH بیت	داده خروجی خوانده‌شده از صف
full	خروجی	۱ بیت	نشان‌دهنده پر بودن صف
empty	خروجی	۱ بیت	نشان‌دهنده خالی بودن صف

رفتار مورد انتظار

¹³First-In First-Out, FIFO

¹⁴Data Buffer

¹⁵Write Interface

¹⁶Read Interface

¹⁷Circular Buffer

¹⁸Pointer

۱. با فعال شدن rst در لبه بالارونده ساعت، اشاره گر خواندن، اشاره گر نوشتن، خروجی داده و وضعیت داخلی صف باید مقداردهی اولیه شوند.
۲. پس از بازنشانی^{۱۹}، صف باید خالی باشد؛ یعنی $empty = 1$ و $full = 0$.
۳. اگر $wr_en = 1$ باشد و صف پر نباشد، مقدار din باید در خانه مشخص شده توسط اشاره گر نوشتن ذخیره شود و اشاره گر نوشتن افزایش یابد.
۴. اگر $rd_en = 1$ باشد و صف خالی نباشد، داده موجود در خانه مشخص شده توسط اشاره گر خواندن باید روی dout قرار گیرد و اشاره گر خواندن افزایش یابد.
۵. اگر صف خالی باشد و خواندن درخواست شود، عملیات خواندن نباید انجام شود و empty باید فعال باقی بماند.
۶. اگر صف پر باشد و نوشتن درخواست شود، عملیات نوشتن نباید انجام شود و full باید فعال باقی بماند.
۷. اگر wr_en و rd_en همزمان برابر ۱ باشند و صف نه خالی و نه پر باشد، هر دو عملیات خواندن و نوشتن باید در همان سیکل انجام شوند و تعداد داده‌های موجود در صف تغییر نکند.
۸. اگر صف خالی باشد و خواندن و نوشتن همزمان درخواست شوند، فقط نوشتن معتبر است.
۹. اگر صف پر باشد و خواندن و نوشتن همزمان درخواست شوند، فقط خواندن معتبر است.
۱۰. داده‌ها باید دقیقاً به همان ترتیبی که وارد شده‌اند از صف خارج شوند.

مطلوب است

۱. پیمانه اصلی صف هم‌گاه را با زبان Verilog طراحی کنید.
۲. میانگیر داخلی صف را به صورت آرایه ثباتی دوبعدی پیاده‌سازی کنید.
۳. اشاره گر خواندن و اشاره گر نوشتن را پیاده‌سازی کرده و نحوه افزایش آن‌ها را در حالت‌های مختلف توضیح دهید.
۴. یک میز آزمون کامل بنویسید که دست‌کم حالت‌های زیر را بررسی کند.

(آ) بازنشانی هم‌گاه مدار؛

(ب) نوشتن چند داده متوالی در صف؛

(ج) خواندن چند داده متوالی از صف؛

(د) بررسی حفظ ترتیب اولین ورودی—اولین خروجی؛

(ه) پر شدن کامل صف و فعال شدن full؛

(و) تلاش برای نوشتن در صف پر؛

(ز) خالی شدن کامل صف و فعال شدن empty؛

(ح) تلاش برای خواندن از صف خالی؛

(ط) خواندن و نوشتن همزمان در حالت عادی؛

(ی) خواندن و نوشتن همزمان در حالت خالی بودن صف؛

(ک) خواندن و نوشتن همزمان در حالت پر بودن صف.

۵. خروجی متنی شبیه‌سازی، تصویر موج‌نگار، گزارش توضیح کد و نتایج را ارائه کنید.

¹⁹Reset

سؤال چهارم: طراحی محاسبه گر سریال CRC-8 (نمره ۴۵)

در این سؤال باید یک پیمان محاسبه گر افزونگی حلقوی^{۲۰} طراحی و پیاده سازی کنید. داده ورودی به صورت سریال و بیت به بیت وارد مدار می شود و مدار باید پس از دریافت کامل پیام، مقدار نهایی CRC را تولید کند.

در این طراحی، مسیر داده باید به صورت کاملاً ساختاری پیاده سازی شود؛ یعنی ثابت CRC باید با نمونه سازی دستی هشت عدد D Flip-Flop و گیت های XOR ساخته شود. استفاده از عملگرهای شیفت رفتاری برای پیاده سازی مسیر داده مجاز نیست. در مقابل، کنترل کننده مرکزی باید به صورت رفتاری و با استفاده از ماشین حالت متناهی طراحی شود.

استاندارد مورد استفاده در این سؤال، CRC-8-ATM با چند جمله ای زیر است:

$$P(x) = x^8 + x^2 + x + 1$$

این چند جمله ای متناظر با مقدار دودویی 0000_0111 یا مقدار شانزده شانزدهی 8'h07 است. مقدار اولیه ثابت CRC در ابتدای هر محاسبه باید برابر 8'h00 باشد. داده ها باید به صورت MSB-first^{۲۱} وارد مدار شوند؛ یعنی بیت پرارزش تر پیام زودتر به مدار داده می شود.

فرض کنید ثابت داخلی با نام crc_reg[7:0] تعریف می شود و بیت [7] crc_reg بیت پرارزش ثابت است. در هر مرحله پذیرش یک بیت داده، سیگنال بازخورد به صورت زیر تعریف می شود:

$$feedback = data_in \oplus crc_reg[7]$$

به روزرسانی ثابت CRC باید دقیقاً مطابق روابط زیر انجام شود:

```
1 crc_next[7] = crc_reg[6];
2 crc_next[6] = crc_reg[5];
3 crc_next[5] = crc_reg[4];
4 crc_next[4] = crc_reg[3];
5 crc_next[3] = crc_reg[2];
6 crc_next[2] = crc_reg[1] ^ feedback;
7 crc_next[1] = crc_reg[0] ^ feedback;
8 crc_next[0] = feedback;
```

مشخصه ها و رابط های ورودی و خروجی

پیمان سطح بالا باید دقیقاً دارای پورت های زیر باشد.

نام پورت	جهت	پهنای	توضیح
clk	ورودی	۱ بیت	ساعت اصلی سامانه
rst_n	ورودی	۱ بیت	بازنشانی ناهم گاه و فعال-پایین
start	ورودی	۱ بیت	سیگنال شروع محاسبه برای یک پیام جدید
data_in	ورودی	۱ بیت	ورودی سریال داده خام پیام
msg_length	ورودی	۱۶ بیت	طول پیام ورودی بر حسب بیت
crc_out	خروجی	۸ بیت	مقدار نهایی محاسبه شده CRC
valid	خروجی	۱ بیت	نشان دهنده معتبر بودن مقدار خروجی crc_out

²⁰Cyclic Redundancy Check (CRC)

²¹Most Significant Bit First

سیگنال‌های کنترلی داخلی پیشنهادی میان کنترل‌کننده و مسیر داده به‌صورت زیر هستند:

نام سیگنال	جهت نسبت به مسیر داده	پهنای	توضیح
en_shift	ورودی	۱ بیت	مجوز پردازش یک بیت جدید و به‌روزرسانی ثابت CRC
sync_clr	ورودی	۱ بیت	پاک‌سازی هم‌گاه ثابت CRC پیش از شروع فریم جدید
crc_value	خروجی	۸ بیت	مقدار فعلی ثابت CRC برای اتصال به خروجی سطح بالا

رفتار مورد انتظار

۱. با فعال شدن $rst_n = 0$ ، کنترل‌کننده و مسیر داده باید به حالت اولیه بازگردند و valid غیرفعال شود.
۲. پس از آزاد شدن بازنشانی، سامانه باید در حالت انتظار باقی بماند تا start فعال شود.
۳. با فعال شدن start، ثابت CRC باید با مقدار $8'h00$ مقداردهی اولیه شود و شمارنده تعداد بیت‌های پردازش شده از صفر آغاز گردد.
۴. در حالت محاسبه، در هر لبه بالارونده ساعت، یک بیت از ورودی data_in پذیرفته شده و مسیر داده مقدار ثابت CRC را مطابق روابط داده‌شده به‌روزرسانی کند.
۵. تعداد بیت‌هایی که باید پردازش شوند توسط msg_length تعیین می‌شود. بنابراین مدار باید پس از پردازش دقیقاً msg_length بیت، محاسبه را متوقف کند.
۶. پس از پایان دریافت پیام، مقدار نهایی ثابت CRC باید روی crc_out قرار گیرد و valid فقط به‌مدت یک سیکل ساعت فعال شود.
۷. پس از پایان مرحله اعتبار خروجی، سامانه باید دوباره به حالت انتظار بازگردد و آماده دریافت پیام بعدی باشد.
۸. اگر start هنگام مشغول بودن سامانه دوباره فعال شود، نباید محاسبه جاری را خراب کند. در گزارش خود مشخص کنید که این سیگنال در حالت‌های غیر از IDLE نادیده گرفته شده یا طبق سیاست مشخص دیگری مدیریت شده است.
۹. اگر $msg_length = 0$ باشد، سامانه نباید هیچ بیت داده‌ای را پردازش کند و باید مقدار اولیه CRC یعنی $8'h00$ را به‌عنوان خروجی معتبر تولید کند.

ساختار پیشنهادی کنترل‌کننده

کنترل‌کننده رفتاری می‌تواند شامل حالت‌های زیر باشد:

IDLE → DONE → SHIFT → CLEAR → IDLE

- در حالت IDLE، سامانه منتظر فعال شدن start است.
- در حالت CLEAR، ثابت CRC به مقدار اولیه $8'h00$ بازگردانده می‌شود و شمارنده بیت‌ها صفر می‌شود.
- در حالت SHIFT، بیت‌های ورودی یکی‌یکی پردازش می‌شوند و شمارنده بیت‌ها افزایش می‌یابد.

• در حالت DONE، خروجی `crc_out` معتبر می‌شود و سیگنال `valid` به مدت یک سیکل ساعت فعال می‌ماند.

وجود حالت CLEAR باعث می‌شود شروع هر فریم جدید مستقل از مقدار باقی‌مانده از فریم قبلی باشد و ابهام زمانی میان دریافت `start` و صفر شدن ثبات CRC از بین برود.

مطلوب است

۱. پیمانۀ D Flip-Flop را طراحی کنید. این پیمانۀ باید دارای ورودی ساعت، بازنشانی ناهم‌گاه فعال-پایین، پاک‌سازی هم‌گاه، مجوز به‌روزرسانی و ورودی داده باشد.

۲. مسیر داده CRC را به صورت کاملاً ساختاری طراحی کنید. در این مسیر داده باید هشت نمونه از D Flip-Flop و گیت‌های XOR مورد نیاز نمونه‌سازی شوند.

۳. روابط بازخورد و سیم‌کشی چندجمله‌ای را دقیقاً مطابق چندجمله‌ای CRC-8-ATM پیاده‌سازی کنید.

۴. کنترل‌کننده رفتاری را با استفاده از ساختار `always` و دستور `case` پیاده‌سازی کنید.

۵. پیمانۀ سطح بالا را طراحی کنید؛ به گونه‌ای که کنترل‌کننده رفتاری و مسیر داده ساختاری در آن نمونه‌سازی شده باشند و نام پورت‌های سطح بالا دقیقاً مطابق جدول باشد.

۶. یک میز آزمون کامل بنویسید که دست‌کم حالت‌های زیر را بررسی کند.

(آ) بازنشانی ناهم‌گاه و فعال-پایین؛

(ب) شروع محاسبه با `start`؛

(ج) پردازش یک پیام با طول غیرصفر؛

(د) پردازش حداقل دو پیام با طول‌های متفاوت؛

(ه) بررسی فعال بودن `valid` فقط برای یک سیکل؛

(و) بررسی آماده شدن مدار برای پیام بعدی؛

(ز) بررسی حالت `msg_length = 0`؛

(ح) بررسی نادیده گرفته شدن یا مدیریت مشخص `start` هنگام مشغول بودن مدار؛

(ط) مقایسه مقدار `crc_out` با مقدار مورد انتظار که در میز آزمون محاسبه یا از پیش تعیین شده است.

۷. خروجی متنی شبیه‌سازی، تصویر موج‌نگار، جدول حالت کنترل‌کننده، توضیح سیم‌کشی مسیر داده و گزارش تحلیل نتایج را ارائه کنید.